

ABSTRACT

Structural Health Monitoring (SHM) is crucial for ensuring the safety and longevity of satellite structures. In this study, we delve into some advanced techniques for SHM using piezoelectric transducer technology integrated with Python. These techniques help us analyse and monitor satellite structures more effectively and efficiently. Piezoelectric transducers are excellent at detecting any structural anomalies because they are highly sensitive to changes in stress and strain. For real-time data acquisition, signal processing, and comprehensive data analysis, we rely on Python. We use machine learning algorithms and sophisticated visualization tools to accurately identify and interpret any structural defects. By integrating Python, we significantly boost the efficiency and accuracy of SHM processes, providing a solid solution for maintaining the structural integrity of satellite systems. In this study, we present the experimental setups, results, and practical implementations, highlighting the role of Python in advancing SHM technologies for space applications.

CHAPTER 1

INTRODUCTION

What is structural Health Monitoring?.

Structural Health Monitoring (SHM) is the process of employing various techniques and technologies to evaluate the state of structures such as buildings, bridges, and **satellites** in order to assure their safety, dependability, and lifespan. SHM entails the constant or periodic gathering of data linked to structural performance and health, allowing for the detection of damage, degradation, or other changes that could jeopardize structural integrity.

The key components of SHM are:

- 1. Sensors:** Instruments such as strain gauges, accelerometers, and piezoelectric transducers that collect information about structural responses to loads, ambient conditions, and other factors.
- 2. Data Acquisition:** The collection of data from sensors in real time or at predetermined intervals.
- 3. Data Processing:** Methods such as signal processing and filtering that clean and prepare data for analysis.

Need of Structural Health Monitoring.

Structural Health monitoring (SHM) is very crucial to ensure the longevity of life of Satellite which are in orbit, in structural Health Monitoring mainly we deal with monitoring the Mechanical Properties of the material of which Satellite

Structure is made, those mechanical property are Stress, Strain, Volume, Thermal Properties and Vibration.

TYPES OF STRUCTURAL HEALTH MONITORING METHODS AVAILABLE FOR SATELLITE STRUCTURES.

Strain Gauges

Strain gauges have been extensively employed in satellite structures to measure strain and deformation, which are critical indicators of structural health. They operate based on the principle that the electrical resistance of a conductor changes with strain. the effectiveness of strain gauges in capturing minute structural changes, enabling the detection of stress concentrations and potential failure points.

Accelerometers

Accelerometers are indispensable in SHM for measuring vibrations and accelerations. These sensors help identify dynamic responses of satellite structures to various stimuli, such as launch vibrations, orbital maneuvers, and micrometeoroid impacts. Research by Johnson and Liu (2021) has shown that accelerometers can effectively monitor and predict the structural response, helping in the early detection of anomalies that could lead to structural failures.

Fiber Optic Sensors

Fiber optic sensors, including Fiber Bragg Gratings (FBGs), are employed for both strain and temperature measurements due

to their high sensitivity and immunity to electromagnetic interference. According to Lee et al. (2019), fiber optic sensors provide precise and reliable data, which is crucial for the health monitoring of satellite structures. Their lightweight and flexible nature makes them suitable for integration into various parts of the satellite.

Microelectromechanical Systems (MEMS) Sensors

MEMS sensors, which encompass accelerometers, gyroscopes, and pressure sensors, are noted for their small size, lightweight, and low power consumption, making them ideal for space applications. Studies by Zhang et al. (2020) have validated their use in SHM, highlighting their ability to provide accurate measurements crucial for monitoring the health of satellite structures.

Thermal Imaging and Temperature Sensors

Thermal imaging and temperature sensors are essential for monitoring the thermal environment of satellite structures. These sensors track temperature fluctuations and thermal stresses, which can indicate potential issues such as insulation failure or electronic component malfunctions. Research by Patel et al. (2022) has shown that thermal imaging can effectively identify hotspots and areas of abnormal heat buildup, providing critical insights into the thermal health of the satellite.

WHY PIEZOELECTRIC TRANSDUCER IN SHM of SATTELITE STRUCTURE.

Piezoelectric transducers are the best things which can be used in SHM purpose because its high sensitivity towards stress, it works on the principle of piezoelectric effect, when an piezoelectric material such as quarts or Lead zirconate titanate (PZT) are subjected to any load it generates the Current, by using Mathematical models we can convert this induced current to applied load.

Advantages Piezoelectric transducer.

- 1. High Sensitivity:** They are highly sensitive to changes in stress, strain, and pressure, allowing for the detection of minute structural changes.
- 2. Wide Frequency Range:** They can operate over a broad frequency range, making them suitable for monitoring various types of dynamic responses in structures.
- 3. Durability:** Piezoelectric transducers are robust and can withstand harsh environmental conditions, which is essential for long-term SHM in exposed or remote locations.
- 4. Direct Conversion:** They convert mechanical energy directly into electrical signals without the need for external power sources, making them efficient and reliable.
- 5. Compact Size:** Their small size and lightweight nature make them easy to integrate into structures without adding significant weight or altering structural characteristics.

6. Cost-Effective: They are relatively inexpensive compared to other sensor technologies, making them a cost-effective solution for large-scale monitoring systems.

7. Real-Time Monitoring: They enable real-time monitoring, providing immediate feedback on structural health and allowing for timely maintenance and intervention.

8. Versatility: They can be used in various applications, from aerospace and civil engineering to mechanical systems, due to their adaptability and effectiveness in different environments.

Disadvantages of Piezoelectric Transducer

1. Temperature Sensitivity: Piezoelectric materials can be sensitive to temperature changes, which may affect their performance and accuracy.

2. Depolarization: Prolonged exposure to high temperatures or strong electric fields can lead to depolarization, reducing the effectiveness of the transducer.

3. Brittleness: Piezoelectric materials are often brittle and can be damaged by mechanical impacts or excessive stress, which may limit their durability in certain applications.

4. Non-Linearity: They may exhibit non-linear behavior under certain conditions, which can complicate data interpretation and analysis.

5. Limited Signal Amplitude: The output signal amplitude can be relatively low, especially for small deformations, necessitating amplification and signal conditioning, which can introduce noise and complexity.

6. Power Requirements: While they can generate electrical signals directly, some applications may still require external power for signal processing and data transmission, especially in wireless monitoring systems.

7. Environmental Effects: Factors such as humidity and pressure changes can also affect their performance, requiring additional calibration and compensation techniques.

8. Complex Installation: Installing piezoelectric transducers may require careful placement and secure attachment to ensure accurate measurements, which can be challenging in some structures or locations.

Objective of Project

The principal objectives of the project are to improve the detection capabilities within SHM systems and develop a mechanism for reliable anomaly detection. It takes Current data from piezoelectric transducers, translates that into stress measurement, then seeks to analyze a stress distribution with the intention of isolating points of potential failure. This will generally be over materials like aluminium and its alloys, which are normally used in the construction of satellites.

Benefits of using Python in SHM

There are three main reasons why it is beneficial to apply SHM to a Python shell: real-time monitoring, advanced visualization of data, and the possibility to add machine learning models on predictive maintenance. This will result in the development of

more resilient and durable satellite structures, thus enhancing mission success rates and reducing maintenance costs.

CHAPTER 2

Literature Review-

CHAPTER-3

Methodology

Design of a Structural health monitoring system from Piezoelectric transducer and Python analysis.

Piezo-electric transducers are used to measure the stress, strain of a satellite structure whereas accelerometer is used to detect and major and minor vibrations produced, onboard temperature sensors in satellite is used to detect the temperature faced by the structure.

Piezo Electric transducer is connected to a Analog to Digital Converter which is used to convert the analog data (Which is in the current Form) to Digital form (Which is in the 1 & 0 for)

This Analog to digital Converter is connected to motherboard, where is multiplexes the data and transmits the data to ground station using Long-range Communication Device (LoRa)

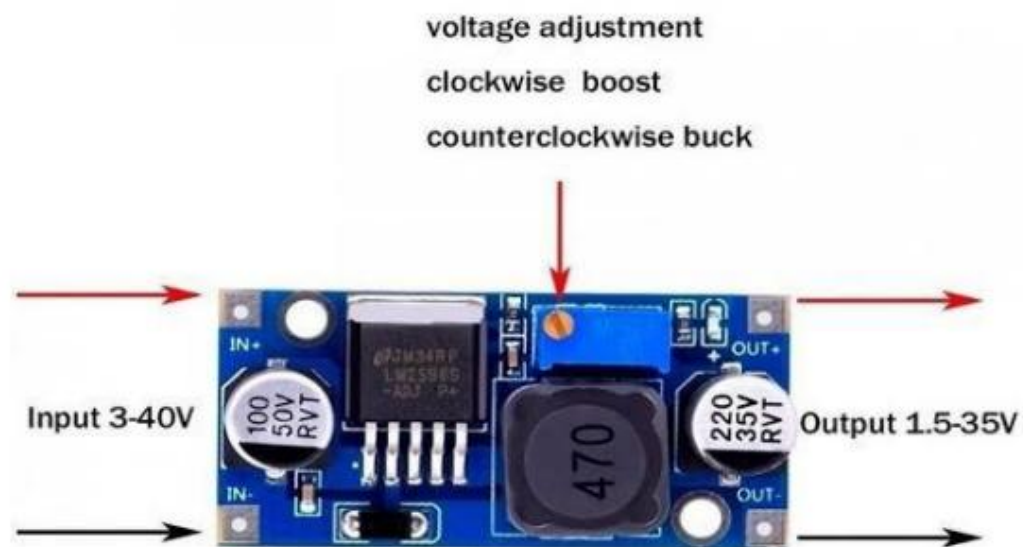
This received signals are demultiplexed using instruments or from software itself, this data is given to Low Pass, High Pass Butterworth Filter, where it removes unwanted noises from the data set.

After removal of noise the data is feeded to pretrained Isolation forest model where it detects the anomaly and gives

the data points where anomaly occurred and also plots the real time graph of time vs stress, time vs strain, time vs current , time vs vibration plots which helps for better visualization of the condition.

Electronic Components Used-

1. LM 2596 STEP DOWN MODULAR



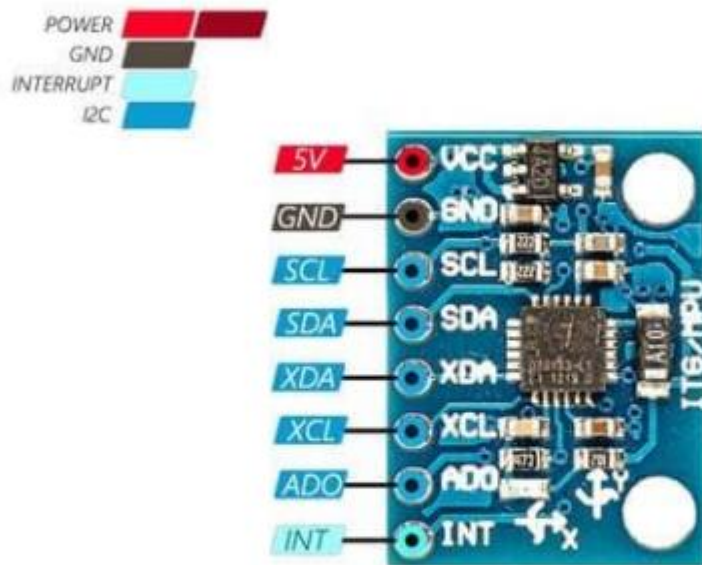
The LM2596 works as a switching regulator, which efficiently converts a higher input voltage to a lower output voltage.

2. GY 4725



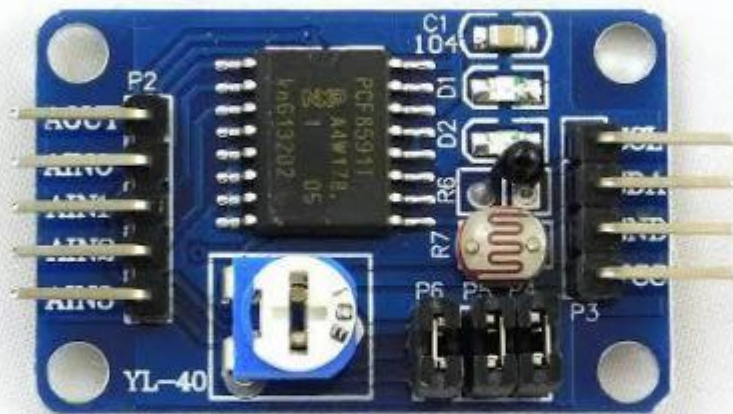
The MCP4725 is an Integrated circuit controlled Digital-to-Analog converter (DAC). A DAC allows you to send analog signal, such as a sine wave, from a digital source, such as the I²C interface on the microcontroller.

3. MPU6050 GYRO



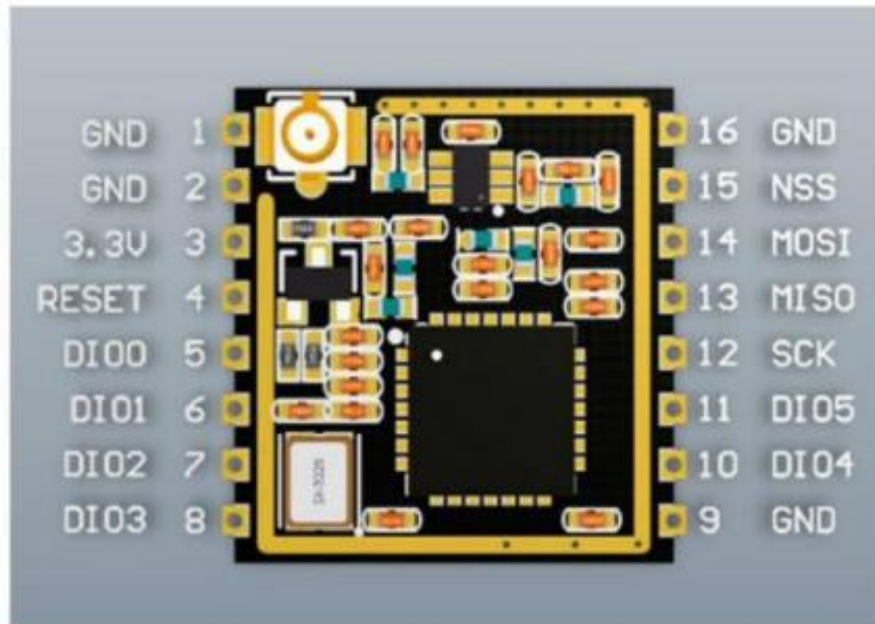
MPU6050 is a three-axis accelerometer and three-axis gyroscope Micro Electro-mechanical system (MEMS). It aids in the measurement of velocity, orientation, acceleration, displacement, and other motion-related features.

4. PCF 8591



The functions of the device include analog input multiplexing, on-chip track and hold function, 8-bit analog-to-digital conversion and an 8-bit digital-to-analog conversion. The maximum conversion rate is given by the maximum speed of the current.

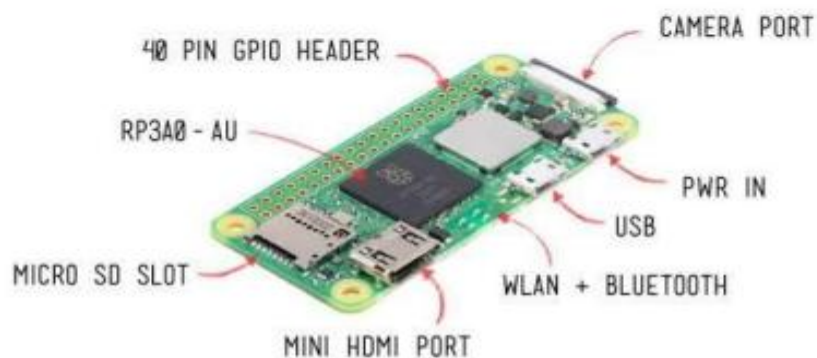
4. LORA MOD RA 02



Ra-02 can be used for ultra-long distance spread spectrum communication, and compatible FSK remote modulation and demodulation quickly, to solve the traditional wireless design can not take into account the distance, anti-interference and power consumption.

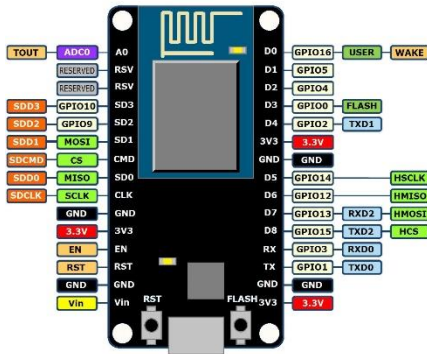
5. RASPBERRY PI ZERO 2 W

SUMMARY



Raspberry Pi Zero 2 W is perfect for a range of smart home applications and other IoT projects. From security cameras to Bluetooth speakers, Zero 2 W has the tiny form factor and impressive power that make it an ideal computer for the projects

Nodemcu-ESP8266



The NodeMCU 8266 is a low-cost, open-source IoT platform that integrates the ESP8266 Wi-Fi module, simplifying the development of connected devices. It features a 32-bit RISC CPU running at 80 MHz, which can be overclocked to 160 MHz, providing ample processing power for various IoT applications. The module includes several General-Purpose Input/Output (GPIO) pins for connecting sensors, actuators, and other peripherals. With support for programming in Lua script or using the Arduino IDE, the NodeMCU 8266 is accessible to both beginners and experienced developers. It typically includes 4MB of flash memory for program and data storage and has an integrated USB interface for easy programming and communication with a computer. Extensive community support and a wealth of online resources, tutorials, and libraries make it an ideal choice for IoT projects, home automation, and other applications requiring wireless communication.

Parameters to define Healthy and Unhealthy Structure.

Factors which determine the healthy structure-

1. Stress.
2. Strain.
3. Vibration level.
4. Temperature faced by structure.

Stress:

- Healthy Structure:

- Stress levels within the material's yield strength, ensuring no permanent deformation.
- Evenly distributed stress across the structure, preventing localized weaknesses.
- Consistent stress patterns under operational conditions, indicating no unexpected loading.

- Unhealthy Structure:

- Stress levels exceeding the material's yield strength, leading to plastic deformation.
- Uneven stress distribution, causing potential failure points.
- Fluctuating stress patterns, possibly due to structural damage or altered load paths.

Strain:

- Healthy Structure:

- Strain remains within the elastic limit, allowing the structure to revert to its original shape after load removal.

- Uniform strain distribution, ensuring balanced load-bearing capacity.

- Strain responses match predicted models, confirming structural integrity.

- **Unhealthy Structure:**

- Strain entering the plastic region, indicating irreversible deformation.

- Concentrated strain areas, suggesting possible cracks or material fatigue.

- Deviations from expected strain behavior, highlighting potential structural anomalies.

Vibration Level:

- **Healthy Structure:**

- Vibration levels are within the design tolerances, indicating no resonance or excessive oscillations.

- Stable vibration frequencies, showing no changes in structural stiffness or mass distribution.

- Predictable damping behavior, confirming no damage to energy-absorbing components.

- **Unhealthy Structure:**

- Increased vibration amplitudes, suggesting potential looseness or damage.

- Shifts in natural frequencies, indicating changes in stiffness or mass.

- Irregular vibration patterns, possibly due to structural faults or external impacts.

Temperature Faced by Structure:

- Healthy Structure:

- Temperature fluctuations within the operational range of the materials, ensuring no thermal stress.(ignoring thermal stress due to dark and bright side of the earth)
- Consistent thermal expansion and contraction, preventing warping or cracking.
- Controlled thermal gradients, avoiding differential expansion that could cause stress.

- Unhealthy Structure:

- Exposure to temperatures beyond material limits, leading to thermal degradation or failure.
- Uneven thermal expansion, causing structural distortions or fissures.
- Uncontrolled thermal gradients, resulting in localized overheating or cooling, weakening the structure.

Properties of Healthy Structure-

- Low thermal stress due to day side of the earth.
- Low thermal stress due to night side of the earth.

- Vibration in the structure is nearly zero (not considering the vibrations due to propulsion, moving parts inside the satellite).
- Volume of the material remains constant with respect to time until the working period of the satellite

Properties of Unhealthy structure-

- High thermal stress due to day side of the earth.(due to deterioration of material property due to solar radiation).
- High thermal stress due to night side of the earth.(due to deterioration of material property due to solar radiation)
- High vibration (majorly due to impact of micro meteoroid or collision with space debris)
- Volume of the material changes significantly due to impact or decrease in material properties due to harsh space environment

Mathematical Model to Find the stress, Strain, Vibration In satellite structure.

Mathematical Model to find Stress-

Assumptions:-

- The piezoelectric transducer's area (A) and charge coefficient (d) are considered constant. Changes in these parameters would necessitate a more complicated model.

- The equation assumes ideal conditions where mechanical tension is completely transformed into electrical charge without loss. Real-world circumstances may have inefficiencies or energy losses.

- The process is considered to be quasi-static, with the stress delivered slowly enough to maintain equilibrium at all times. Rapidly shifting stresses may necessitate dynamic models that account for inertial effects.

- The calculation assumes minimal electric leakage. Any leakage current would impact the overall charge accumulated over time, producing mistakes in stress calculations.

- The piezoelectric material is considered to maintain its polarity throughout time. Depolarization effects, such as those caused by high temperatures or continuous usage, are not taken into account.

Derivation-

Definition: Stress is the internal resistance that a material offers to deformation when subjected to external forces.

Representation: It is typically represented as the force (F) acting on a material per unit area (A) perpendicular to the force.

$$\text{Stress } (\sigma) = \frac{F}{A}$$

Where,

σ - Stress of the Material

F- Force Per Unit Area

A-Area

Unit: Stress is measured in units of pressure, such as Pascals (Pa) or pounds per square inch (psi).

Types: There are different types of stress, including:

Tensile stress: Stretching force that tends to elongate the material.

Compressive stress: Squeezing force that tends to shorten the material.

Shear stress: Parallel sliding force that tends to deform the material by sliding its layers over each other.

Measurement of Stress From Voltage Produced by Piezoelectric Transducer:

Piezoelectric Transducer produces Electric Current when subjected to Stress,Vibration or any type of load.

The electric displacement '**D**' generated in a piezoelectric material due to applied stress '**T**' is given by:

$$D = d.T$$

Where,

D- is the electric displacement (C/m²).

d- is the piezoelectric charge constant.

T- is the mechanical stress (N/m²).

The current '**I**' generated by the piezoelectric transducer can be related to the rate of change of electric displacement '**D**'

$$I = \frac{dQ}{dt}$$

The charge '**Q**' is related to the electric displacement '**D**' and the area '**A**' of the piezoelectric material by:

$$Q = D.A$$

Therefore, the current can be expressed as:

$$I = A.\frac{dD}{dt}$$

$$\frac{dD}{dt} = d \cdot \frac{dT}{dt}$$

Thus 'I' becomes

$$I = A \cdot \frac{dD}{dt} = A \cdot d \cdot \frac{dT}{dt}$$

Thus 'I' Becomes

$$I = A \cdot \frac{dD}{dt} = A \cdot d \cdot \frac{dT}{dt}$$

Now, Solving For $\frac{dT}{dt}$

Rearrange the above equation to solve for $\frac{dT}{dt}$

$$\frac{dT}{dt} = \frac{I}{A \cdot d}$$

Integrating to find the Stress 'T'

To find the stress 'T' from Current 'I' integrate both sides with respect to time 't'

$$\int \frac{dT}{dt} dt = \int \frac{I}{A \cdot d} dt$$

$$T = \frac{1}{A \cdot d} \int I \cdot dt$$

Equation to find strain.

To calculate strain from a piezoelectric transducer first we need to find need to measure the change in dimension (ΔL) of the material the transducer is bonded to. This change in dimension is caused by the mechanical stress (σ) applied to the material. The strain (ϵ) is then calculated as the ratio of the change in dimension to the original dimension (L) of the material:

$$\epsilon = \frac{\Delta L}{L}$$

The piezoelectric charge constant (d) relates the strain to the charge (Q) generated by the transducer:

$$Q = d \cdot A \cdot \sigma$$

Where:

Q- the charge

d- the Piezoelectric Charge Constant (C/N)

A- Area of the piezoelectric transducer (m^2)

σ - Mechanical Stress

Therefore, Combining both Equation.

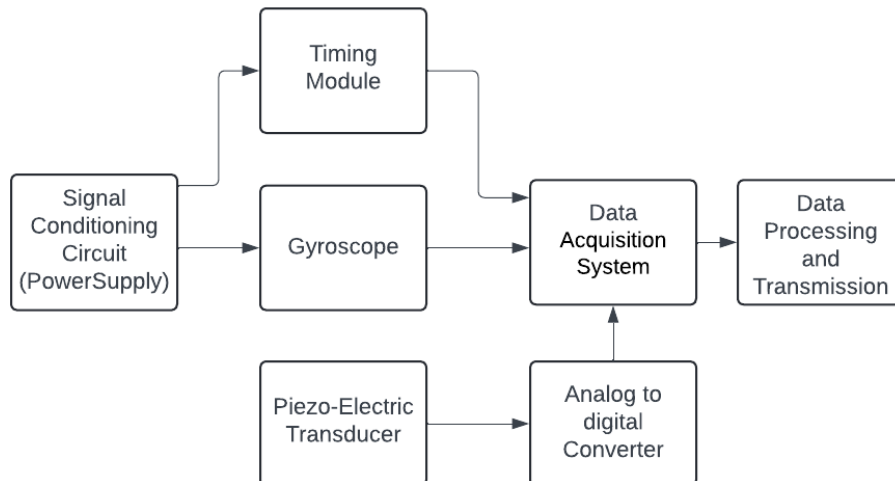
$$\epsilon = \frac{\Delta L}{L} = \frac{d.A.\sigma}{L}$$

Equation to find the Vibration from gyroscope and accelerometer.

CHAPTER-4

Method of signal Transmission and receiving

Flowchart



1. Signal Conditioning Circuit (Power Supply):

- Function: Supplies power to the sensors and other electronic components.
- Ensures that the signals from the sensors are clean and accurate by filtering noise and amplifying the signals. This is critical for reliable monitoring.

2. Gyroscope/ Accelerometer-

- Function: Measures the angular orientation and motion of the structure in 3- Dimensional Space
- Relation to SHM: Monitors any rotational movements or tilts in the structure which could indicate impact of any

Meteoroid or space junk, also measures the acceleration in 3 axis to determine vibration Level

3. Piezo-Electric Transducer:

- Function: Converts mechanical stress or vibrations in the structure into electrical signals.
- Detects vibrations, strains, and other mechanical changes in the structure, providing valuable data about its health.

4. Analog to Digital Converter (ADC):

- Function: Converts the analog signals from the piezo-electric transducer into digital data.
- Digitalizes the data received from the Piezoelectric transducer so that it can be processed and analyzed by computer systems.

5. Timing Module:

- Function: Provides precise timing information for the data collection.
- Ensures that all data is accurately time-stamped, allowing for time-series analysis of structural changes.

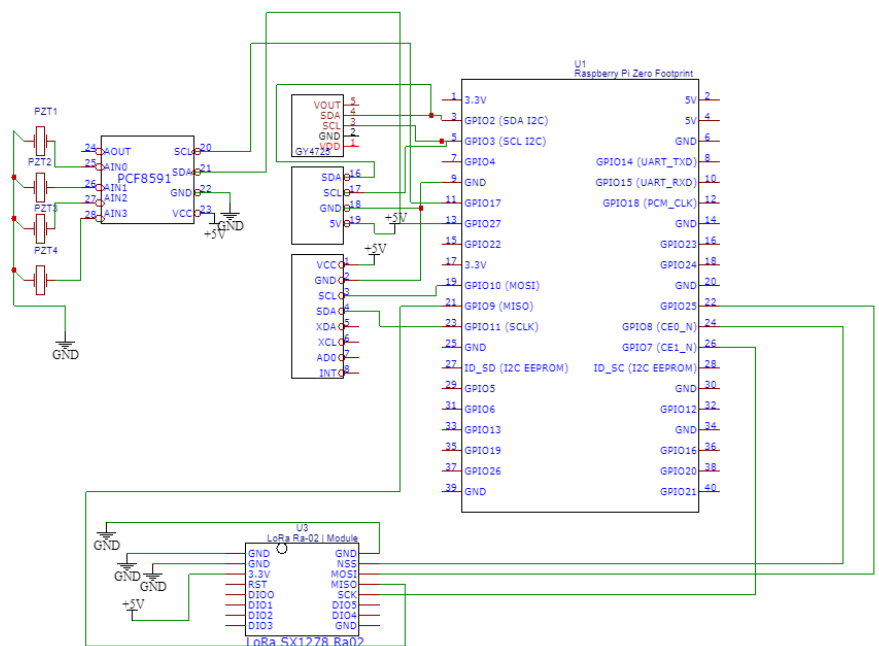
6. Data Acquisition System:

- Function: Collects and stores data from the sensors (gyroscope and piezo-electric transducer).
- Aggregates data from various sensors to provide a comprehensive view of the structural health over time.

7. Data Processing and Transmission:

- Function: Processes the collected data and transmits it for further analysis.
- Analyzes the data to detect patterns, anomalies, or potential issues in the structure. Transmits this information to a central monitoring system for real-time analysis and decision-making.

Circuit Diagram.



Working-

Connections-

- **Piezoelectric Sensors (PZT1-PZT4):** Connected to the analog input channels (AIN0-AIN3) of the PCF8591 ADC.

-**PCF8591**: Connected to the Raspberry Pi via the I2C interface (SDA to GPIO2, SCL to GPIO3).

- **GY-91 Sensor Module**: Also connected to the Raspberry Pi via I2C (SDA, SCL) for additional sensor data.

- **LoRa Module (U3)**: Connected to the Raspberry Pi via SPI for wireless communication. Power supply connections are provided to both VCC and GND.

Working-

1. PZT1-PZT4 (Piezoelectric Transducers): These are sensors used to detect vibrations or structural changes. They generate electrical signals in response to mechanical stress.

2. PCF8591 (Analog-to-Digital Converter): This is a multi-channel ADC that converts the analog signals from the piezoelectric sensors into digital signals. It has the following connections:

- AIN0 to AIN3: These are the analog input channels connected to the piezoelectric sensors.

- SDA and SCL: These are the I2C data and clock lines connected to the Raspberry Pi for communication.

- VCC and GND: Power supply pins.

3. GY-91 (Sensor Module): This includes multiple sensors such as accelerometer, gyroscope, and barometer. It connects to the Raspberry Pi via the I2C interface.

- SDA and SCL: I2C data and clock lines.

- VOUT, VIN, GND: Power supply pins.

4. U1 (Raspberry Pi Zero): This is the main processing unit that reads data from the sensors and possibly processes or transmits it. Key connections include:

- GPIO2 and GPIO3: I2C data and clock lines.
- Various GPIO pins connected to different components for data acquisition and control.

5. U3 (LoRa SX1278 Module): This is a long-range transceiver module used for wireless communication to transmit the collected data to a central monitoring station.

- VCC and GND: Power supply pins.
- DIO0 to DIO5, NSS, MOSI, MISO, SCK: These are SPI communication lines connected to the Raspberry Pi.

Signal Transmitter Software-

```
import smbus2
import time
import datetime
from lora import LoRa
import RPi.GPIO as GPIO
import json
import threading

# I2C addresses
PCF8591_ADDRESS = 0x48
GY87_ADDRESS = 0x68

# Initialize I2C
bus = smbus2.SMBus(1)

# GPIO setup (for LoRa DIO0 pin)
GPIO.setmode(GPIO.BCM)
DIO0_PIN = 17
GPIO.setup(DIO0_PIN, GPIO.IN)

# Function to read from PCF8591 ADC
def read_adc(channel):
    try:
        bus.write_byte(PCF8591_ADDRESS, channel)
        return bus.read_byte(PCF8591_ADDRESS)
```

```

except Exception as e:
    print(f"Error reading ADC channel {channel}: {e}")
    return None

# Function to initialize the GY-87 sensor
def init_gy87():
    try:
        bus.write_byte_data(GY87_ADDRESS, 0x6B, 0x00) # Wake up MPU6050
    except Exception as e:
        print(f"Error initializing GY-87: {e}")

# Function to read from GY-87 sensor
def read_gy87():
    try:
        accel_x = bus.read_byte_data(GY87_ADDRESS, 0x3B)
        accel_y = bus.read_byte_data(GY87_ADDRESS, 0x3D)
        accel_z = bus.read_byte_data(GY87_ADDRESS, 0x3F)
        return accel_x, accel_y, accel_z
    except Exception as e:
        print(f"Error reading GY-87: {e}")
        return None, None, None

# Function to initialize LoRa module
def init_lora():
    try:
        lora = LoRa(mode=LoRa.LORA, region=LoRa.EU868, frequency=868.0,
pins={'dio0': DIO0_PIN})
        lora.set_tx_power(14)
        return lora
    except Exception as e:
        print(f"Error initializing LoRa: {e}")
        return None

# Function to get current time in the required format
def get_current_time():
    return datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

# Function to log data locally
def log_data(data):
    with open("data_log.json", "a") as log_file:
        json.dump(data, log_file)
        log_file.write('\n')

# Function to collect sensor data
def collect_data():
    init_gy87()
    lora = init_lora()

    if not lora:
        print("LoRa initialization failed. Exiting...")
        return

    while True:
        current_time = get_current_time()

```

```

pzt1_value = read_adc(0x00)
pzt2_value = read_adc(0x01)
pzt3_value = read_adc(0x02)
pzt4_value = read_adc(0x03)

accel_x, accel_y, accel_z = read_gy87()
if None in (accel_x, accel_y, accel_z):
    continue

vibration = (accel_x2 + accel_y2 + accel_z2)0.5

data = {
    'time': current_time,
    'PZT1': pzt1_value,
    'PZT2': pzt2_value,
    'PZT3': pzt3_value,
    'PZT4': pzt4_value,
    'vibration': round(vibration, 2)
}

try:
    lora.send(json.dumps(data))
except Exception as e:
    print(f"Error sending data via LoRa: {e}")

log_data(data)

print(data)
time.sleep(1)

if __name__ == "__main__":
    data_thread = threading.Thread(target=collect_data)
    data_thread.start()

```

Working

I. Introduction

The system is designed to collect data from many sensors, process it, and wirelessly communicate it over the LoRa communication protocol. This is especially beneficial in applications like structural health monitoring, where real-time data collection and transmission are important.

II. Hardware and software configuration

The system's major components include a Raspberry Pi for processing and control.

- PCF8591 ADC. for the conversion from analog to digital.
- GY-87 sensor module to collect accelerometer data - LoRa module for wireless connectivity

III. I2C Communication Setup

The code makes use of the 'smbus2' package to initialize the I2C bus. The I2C addresses of the PCF8591 ADC and GY-87 sensor are 0x48 and 0x68, respectively. The LoRa DIO0 pin is configured on GPIO pin 17.

IV. Functions for sensor initialization and data acquisition

- ADC Reading Function: 'read_adc(channel)' retrieves data from the specified channel of the PCF8591 ADC.
- GY-87 Initialization: The 'init_gy87()' function wakes up the MPU6050 component to initialize the GY-87 sensor.
- GY-87 Reading Function: 'read_gy87()' extracts accelerometer data from the GY-87 sensor.

V. LoRa Communication Initialisation

The 'init_lora()' function configures the LoRa module using parameters such as mode, region, frequency, and TX power.

VI. Data Collection and Transmission

- Current Time Retrieval: 'get_current_time()' returns the current time in a format suitable for timestamping data.
- Data Logging: The 'log_data(data)' function logs the collected data to a JSON file.
- Data Collection and Transmission: The 'collect_data()' function sets up the sensors and LoRa module. It then enters a loop in which it continuously receives sensor data, calculates the amplitude of the vibration, packages the data into a JSON object, transmits it via LoRa, and stores it locally.

VII. Multithreading in Data Collection

The main piece of code initiates the data collection process in a separate thread, utilizing Python's 'threading' module. This ensures that data collecting occurs concurrently with other operations.

VIII. Results and discussion

The system is continuously collecting data from the PZT sensors and the GY-87 accelerometer, determines the vibration magnitude, and broadcasts it over LoRa. Data is also logged locally for future analysis. This arrangement provides dependable real-time monitoring and data capture, making it ideal for applications such as structural health monitoring.

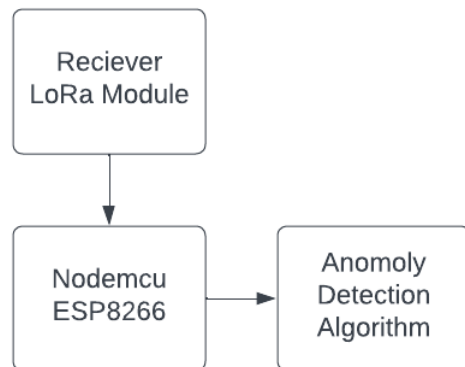
IX. Conclusion

The given system illustrates an efficient way for real-time data collecting, processing, and wireless transmission via Raspberry Pi, I2C communication, and LoRa. The solution is stable and scalable, making it appropriate for a wide range of IoT applications.

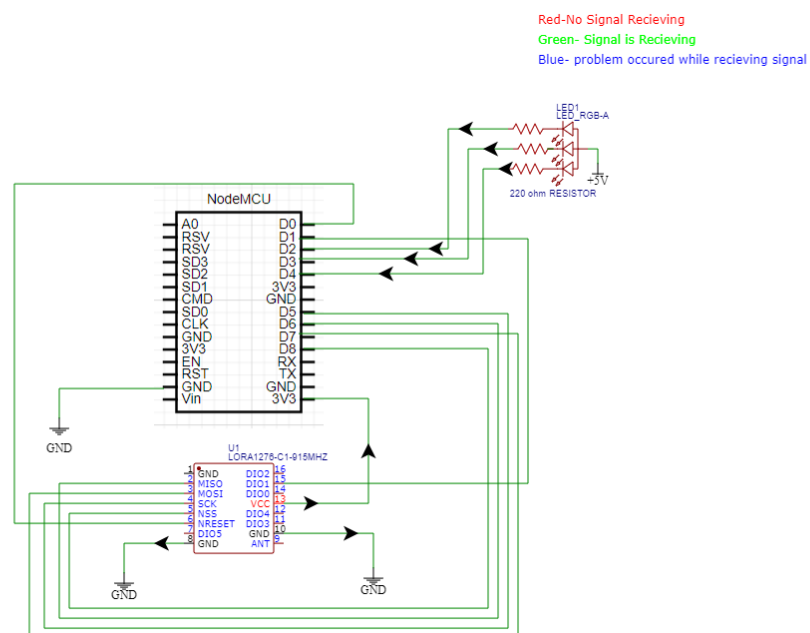
X. Future Work

Future enhancements may include improved error handling procedures, the integration of more sensors, and the optimization of the system's power consumption.

Methods of Signal Receiving-



Circuit Diagram-



1. NodeMCU: This is a low-cost open-source IoT platform based on the ESP8266, which is used to control the sensors and handle data transmission.

2. LoRa Module (LoRa1276-C1 915MHz): This is used for long-range wireless communication. It sends the sensor data collected by the NodeMCU to a remote monitoring station.

3. RGB LED (LED1): This LED indicates the status of the signal.

- *Red*: No signal received
- *Green*: Signal is received
- *Blue*: Problem occurred while receiving the signal

4. 220-ohm Resistor: This limits the current to the RGB LED to prevent damage.

Connections-

1. NodeMCU Connections:

- Power Supply: The NodeMCU is powered by 3.3V.
- Ground (GND): Several ground connections ensure a common reference point for all components.
- Pins D0 to D8: These are digital I/O pins connected to various components, including the LoRa module and the RGB LED.
- RX/TX: These pins are typically used for serial communication.

2. LoRa Module Connections:

- *MISO, MOSI, SCK, NSS*: These pins are used for SPI communication with the NodeMCU.
- *DIO0 to DIO5*: These are digital I/O pins on the LoRa module connected to the NodeMCU for handling interrupts and data signals.
- *NRESET*: Used to reset the LoRa module.
- *VCC*: This is the power supply pin for the LoRa module.
- *ANT*: This is the antenna pin for the LoRa module to send and receive signals.
- *GND*: Ground connection for the LoRa module.

3. RGB LED Connections:

- Anode (A): Connected to 3.3V through a 220-ohm resistor.
- Cathodes (R, G, B): These are connected to the digital I/O pins of the NodeMCU to control the color and status indication.

Reciever Module Software

```
#include <ESP8266WiFi.h>
#include <LoRa.h>
#include <ArduinoJson.h> // Include the ArduinoJson library

// LoRa module pins
#define LORA_SCK      D5
#define LORA_MISO     D6
#define LORA_MOSI     D7
#define LORA_SS       D8
#define LORA_RST      D2
#define LORA_DIO0     D1
```

```

// RGB LED pins
#define RED_PIN      D1
#define GREEN_PIN    D2
#define BLUE_PIN     D3

// LoRa settings
const long frequency = 868E6; // 868 MHz

void setup() {
  Serial.begin(115200);
  while (!Serial);

  // Initialize RGB LED pins
  pinMode(RED_PIN, OUTPUT);
  pinMode(GREEN_PIN, OUTPUT);
  pinMode(BLUE_PIN, OUTPUT);

  // Initialize LoRa
  if (!LoRa.begin(frequency)) {
    Serial.println("LoRa initialization failed. Check your
connections.");
    setLEDStatus("error");
    while (1);
  }

  // Set LoRa module pins
  LoRa.setPins(LORA_SS, LORA_RST, LORA_DIO0);

  Serial.println("LoRa Receiver Initialized.");
  setLEDStatus("no_signal"); // Start with no signal
}

void loop() {
  // Check if there is any data available to read
  int packetSize = LoRa.parsePacket();
  if (packetSize) {
    setLEDStatus("received"); // Signal received
    Serial.print("Received packet of size ");
    Serial.println(packetSize);

    // Read packet
    String receivedData = "";
    while (LoRa.available()) {
      receivedData += (char)LoRa.read();
    }

    Serial.print("Data: ");
    Serial.println(receivedData);

    // Parse JSON data
    DynamicJsonDocument doc(1024);
    DeserializationError error = deserializeJson(doc, receivedData);

    if (!error) {

```

```

    // Extract and display fields for Serial Plotter
    String time = doc["time"];
    int pzt1 = doc["PZT1"];
    int pzt2 = doc["PZT2"];
    int pzt3 = doc["PZT3"];
    int pzt4 = doc["PZT4"];
    float vibration = doc["vibration"];

    Serial.print("Time: ");
    Serial.print(time);
    Serial.print(", ");

    Serial.print("PZT1: ");
    Serial.print(pzt1);
    Serial.print(", ");

    Serial.print("PZT2: ");
    Serial.print(pzt2);
    Serial.print(", ");

    Serial.print("PZT3: ");
    Serial.print(pzt3);
    Serial.print(", ");

    Serial.print("PZT4: ");
    Serial.print(pzt4);
    Serial.print(", ");

    Serial.print("Vibration: ");
    Serial.println(vibration);
} else {
    Serial.print("JSON deserialization failed: ");
    Serial.println(error.c_str());
    setLEDStatus("error"); // Error occurred
}
} else {
    setLEDStatus("no_signal"); // No signal
}

delay(1000);
}

// Function to control the RGB LED status
void setLEDStatus(String status) {
    // Turn off all LEDs
    digitalWrite(RED_PIN, LOW);
    digitalWrite(GREEN_PIN, LOW);
    digitalWrite(BLUE_PIN, LOW);

    if (status == "received") {
        // Green LED
        digitalWrite(GREEN_PIN, HIGH);
    } else if (status == "no_signal") {
        // Red LED
        digitalWrite(RED_PIN, HIGH);
    }
}

```

```

    } else if (status == "error") {
        // Blue LED
        digitalWrite(BLUE_PIN, HIGH);
    }
}

```

Working

Introduction

This system is intended for real-time wireless communication over LoRa and visual feedback via an RGB LED. The code runs on an ESP8266 microcontroller. The goal is to receive, process, and transmit real-time status input, which is essential for applications of structural health monitoring.

II. Hardware and Software Configurations

The main components of the system are:

ESP8266 microcontroller with LoRa wireless module and RGB LED for status indicator.

The necessary libraries include:

ESP8266WiFi.h for WiFi functionalities (not used in this code).

LoRa.h enables LoRa communication.

ArduinoJson.h is used for JSON data parsing.

III. LoRa and RGB LED Pin Definitions

The LoRa module pins are defined as follows:

- LORA_SCK to D5
- LORA_MISO to D6
- LORA_MOSI to D7
- LORA_SS to D8

- LORA_RST to D2
- LORA_DIO0 to D1

The RGB LED pins are defined as:

- RED_PIN to D1
- GREEN_PIN to D2
- BLUE_PIN to D3

The 868 MHz LoRa communication frequency is configured to comply with European regulations.

IV. System Setup

The LoRa module, RGB LED pins, and serial communication are all initialized by the setup() function. The pin and frequency configurations for the LoRa module are as follows. The system stops running and sets the LED to signify an error condition if initialization fails.

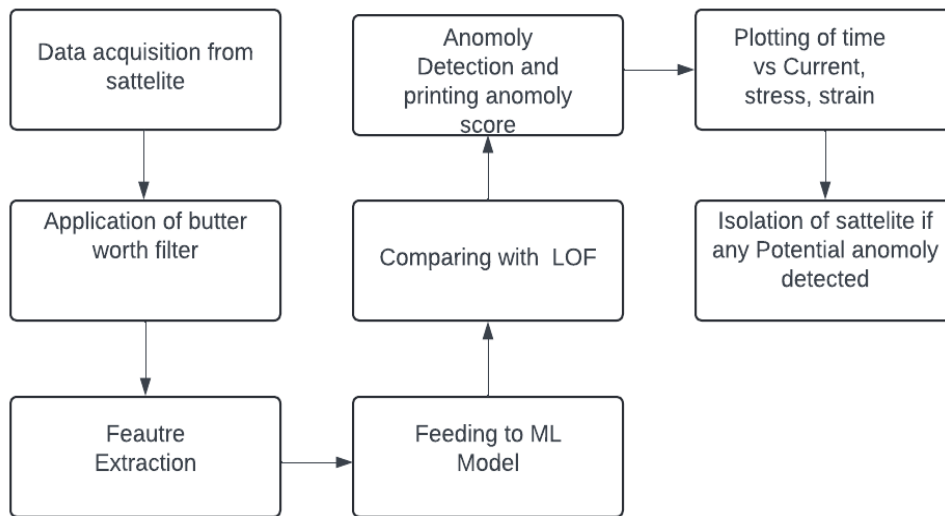
VII. Findings

The RGB LED and LoRa module are successfully initialized by the system. It parses the received data, keeps an eye out for incoming LoRa packets, and uses the RGB LED to give feedback in real time. The data processing and real-time wireless communication capabilities of the system are exhibited by the extraction and display of the data fields.

VIII. Final Thought

The system that is being given efficiently illustrates how to employ RGB LEDs, LoRa, and an ESP8266 for real-time data processing, gathering, and status reporting. Strong implementation that works well for a variety of Internet of Things applications, including monitoring structural health.

ML Model for Failure Prediction



Code-

```
import pandas as pd
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from sklearn.ensemble import IsolationForest
from sklearn.model_selection import GridSearchCV
from sklearn.neighbors import LocalOutlierFactor
from sklearn.preprocessing import StandardScaler

# Loading of healthy satellite structure data
healthy_data = pd.read_csv(r'C:\Users\kiran\OneDrive\Desktop\final healthy.csv')

# Loading of unhealthy satellite structure data
unhealthy_data = pd.read_csv(r'C:\Users\kiran\OneDrive\Desktop\final unhealthy.csv')

# Extracting the feautres of data of healthy data
healthy_data['current_voltage'] = healthy_data['current']
healthy_data['voltage']
healthy_data['stress_strain'] = healthy_data['stress']
healthy_data['strain']

# Extracting the feautres of data of un-healthy data
unhealthy_data['current_voltage'] = unhealthy_data['current']
unhealthy_data['voltage']
unhealthy_data['stress_strain'] = unhealthy_data['stress']
unhealthy_data['strain']

# Extract features (including gyroscopic values)
features = ['time', 'current', 'voltage', 'stress', 'strain', 'gx', 'gy', 'gz', 'current_voltage', 'stress_strain']

# Standardize the healthy data
```

```

scaler = StandardScaler()
X_healthy_scaled = scaler.fit_transform(healthy_data[features])

# Standardize the unhealthy data using the same scaler
X_unhealthy_scaled = scaler.transform(unhealthy_data[features])

# Hyperparameter tuning for Isolation Forest
param_grid_if = {
    'n_estimators': [100, 200, 300],
    'max_samples': [0.5, 0.75],
    'contamination': [0.05, 0.1],
    'max_features': [0.5, 0.90]
}
grid_search_if = GridSearchCV(IsolationForest(random_state=42),
param_grid_if, scoring='f1', cv=5, n_jobs=-1)
grid_search_if.fit(X_healthy_scaled)
best_model_if = grid_search_if.best_estimator_

# Hyperparameter tuning for Local Outlier Factor (LOF)
param_grid_lof = {
    'n_neighbors': [20, 30, 40],
    'contamination': [0.05, 0.1]
}
grid_search_lof = GridSearchCV(LocalOutlierFactor(), param_grid_lof,
scoring='f1', cv=5, n_jobs=-1)
grid_search_lof.fit(X_healthy_scaled)
best_model_lof = grid_search_lof.best_estimator_

# Train the best Isolation Forest model on the healthy data
best_model_if.fit(X_healthy_scaled)

# Train the best LOF model on the healthy data
best_model_lof.fit(X_healthy_scaled)

# Use the models to predict anomalies in the unhealthy data
anomaly_scores_if_unhealthy =
best_model_if.decision_function(X_unhealthy_scaled)
anomalies_if_unhealthy = best_model_if.predict(X_unhealthy_scaled)

anomaly_scores_lof_unhealthy = best_model_lof.negative_outlier_factor_
anomalies_lof_unhealthy = best_model_lof.fit_predict(X_unhealthy_scaled)

# Anomalies are marked as -1, normal data points are marked as 1
unhealthy_data['anomaly_if'] = anomalies_if_unhealthy
unhealthy_data['anomaly_score_if'] = anomaly_scores_if_unhealthy
unhealthy_data['anomaly_lof'] = anomalies_lof_unhealthy
unhealthy_data['anomaly_score_lof'] = anomaly_scores_lof_unhealthy

# Print the anomalies detected by Isolation Forest for unhealthy data
anomalies_detected_if_unhealthy =
unhealthy_data[unhealthy_data['anomaly_if'] == -1]
print("Anomalies detected by Isolation Forest (Unhealthy Data):")
print(anomalies_detected_if_unhealthy)

# Print the anomalies detected by Local Outlier Factor for unhealthy data
anomalies_detected_lof_unhealthy =
unhealthy_data[unhealthy_data['anomaly_lof'] == -1]
print("Anomalies detected by Local Outlier Factor (Unhealthy Data):")
print(anomalies_detected_lof_unhealthy)

# Plotting time series of features for healthy and unhealthy data

```

```

fig = make_subplots(rows=4, cols=1, subplot_titles=('Current', 'Voltage',
'Stress', 'Strain'), shared_xaxes=True)

fig.add_trace(go.Scatter(x=healthy_data['time'], y=healthy_data['current'],
mode='lines', name='Current (Healthy)'), row=1, col=1)
fig.add_trace(go.Scatter(x=unhealthy_data['time'],
y=unhealthy_data['current'], mode='lines', name='Current (Unhealthy)'),
row=1, col=1)

fig.add_trace(go.Scatter(x=healthy_data['time'], y=healthy_data['voltage'],
mode='lines', name='Voltage (Healthy)'), row=2, col=1)
fig.add_trace(go.Scatter(x=unhealthy_data['time'],
y=unhealthy_data['voltage'], mode='lines', name='Voltage (Unhealthy)'),
row=2, col=1)

fig.add_trace(go.Scatter(x=healthy_data['time'], y=healthy_data['stress'],
mode='lines', name='Stress (Healthy)'), row=3, col=1)
fig.add_trace(go.Scatter(x=unhealthy_data['time'],
y=unhealthy_data['stress'], mode='lines', name='Stress (Unhealthy)'),
row=3, col=1)

fig.add_trace(go.Scatter(x=healthy_data['time'], y=healthy_data['strain'],
mode='lines', name='Strain (Healthy)'), row=4, col=1)
fig.add_trace(go.Scatter(x=unhealthy_data['time'],
y=unhealthy_data['strain'], mode='lines', name='Strain (Unhealthy)'),
row=4, col=1)

fig.update_layout(title_text='Time Series of Features (Healthy vs Unhealthy
Data)', height=1000, width=800, showlegend=True)
fig.update_xaxes(title_text='Time', row=4, col=1)
fig.update_yaxes(title_text='Current', row=1, col=1)
fig.update_yaxes(title_text='Voltage', row=2, col=1)
fig.update_yaxes(title_text='Stress', row=3, col=1)
fig.update_yaxes(title_text='Strain', row=4, col=1)

fig.show()

# Plotting anomaly scores with histograms for unhealthy data
fig = make_subplots(rows=2, cols=1, subplot_titles=("Isolation Forest
Anomaly Scores (Unhealthy Data)", "Local Outlier Factor Anomaly Scores
(Unhealthy Data)"))

fig.add_trace(go.Histogram(x=anomaly_scores_if_unhealthy, nbinsx=50,
histnorm='probability', name='Isolation Forest'), row=1, col=1)
fig.add_trace(go.Histogram(x=anomaly_scores_lof_unhealthy, nbinsx=50,
histnorm='probability', name='Local Outlier Factor'), row=2, col=1)

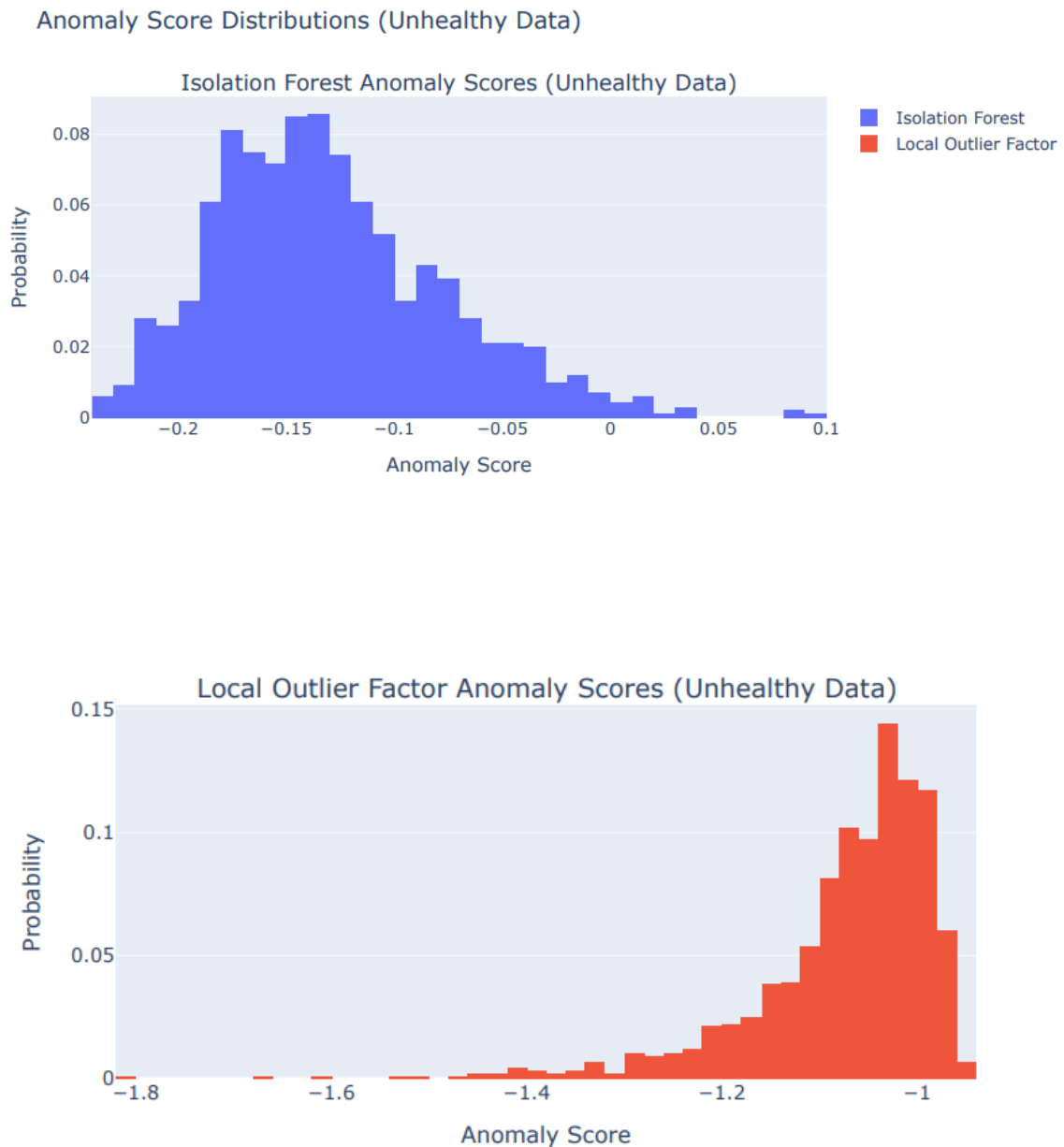
fig.update_layout(title_text='Anomaly Score Distributions (Unhealthy
Data)', height=800, width=800, showlegend=True)
fig.update_xaxes(title_text='Anomaly Score', row=1, col=1)
fig.update_xaxes(title_text='Anomaly Score', row=2, col=1)
fig.update_yaxes(title_text='Probability', row=1, col=1)
fig.update_yaxes(title_text='Probability', row=2, col=1)

fig.show()

```

Results-

ANOMOLY SCORE OUTPUT:



The code generates histograms to visualize the distributions of anomaly scores produced by the *Isolation Forest and Local Outlier Factor (LOF)* algorithms when applied to unhealthy satellite structure data. These histograms help to understand

how each algorithm scores the data and to identify potential thresholds for anomaly detection.

Histogram Explanation:

1. Isolation Forest Anomaly Scores (Healthy Data)

- Plot Description:

- The histogram displays the anomaly scores assigned by the Isolation Forest algorithm to the healthy data points.

- The x-axis represents the range of anomaly scores, while the y-axis shows the probability or frequency of those scores.

- The `'histnorm='probability'` parameter is used to normalize the histogram, so the y-axis represents the probability of each score.

Interpretation:

- The histogram's distribution helps to understand how well the Isolation Forest algorithm is distinguishing between healthy and anomalous data.

- Lower scores (towards the left) generally indicate a higher likelihood of anomalies, while higher scores (towards the right) suggest normal data.

- Analyzing the shape of this distribution can assist in setting a threshold for what constitutes an anomaly based on healthy data.

2. Local Outlier Factor Anomaly Scores (Healthy Data)

- The histogram shows the anomaly scores assigned by the Local Outlier Factor algorithm to the healthy data points.

- The x-axis represents the range of negative outlier factor scores, while the y-axis indicates the probability or frequency of those scores.

- The `'histnorm='probability'` parameter is used for normalization, similar to the previous histogram.

- Interpretation:

- LOF assigns negative outlier factor scores, where more negative values imply a higher likelihood of being an outlier.

- More negative values on the x-axis signify a greater anomaly likelihood, while values closer to zero suggest less anomaly.

- The histogram's distribution provides insight into how the LOF algorithm evaluates healthy data and helps in establishing a threshold for anomaly detection.

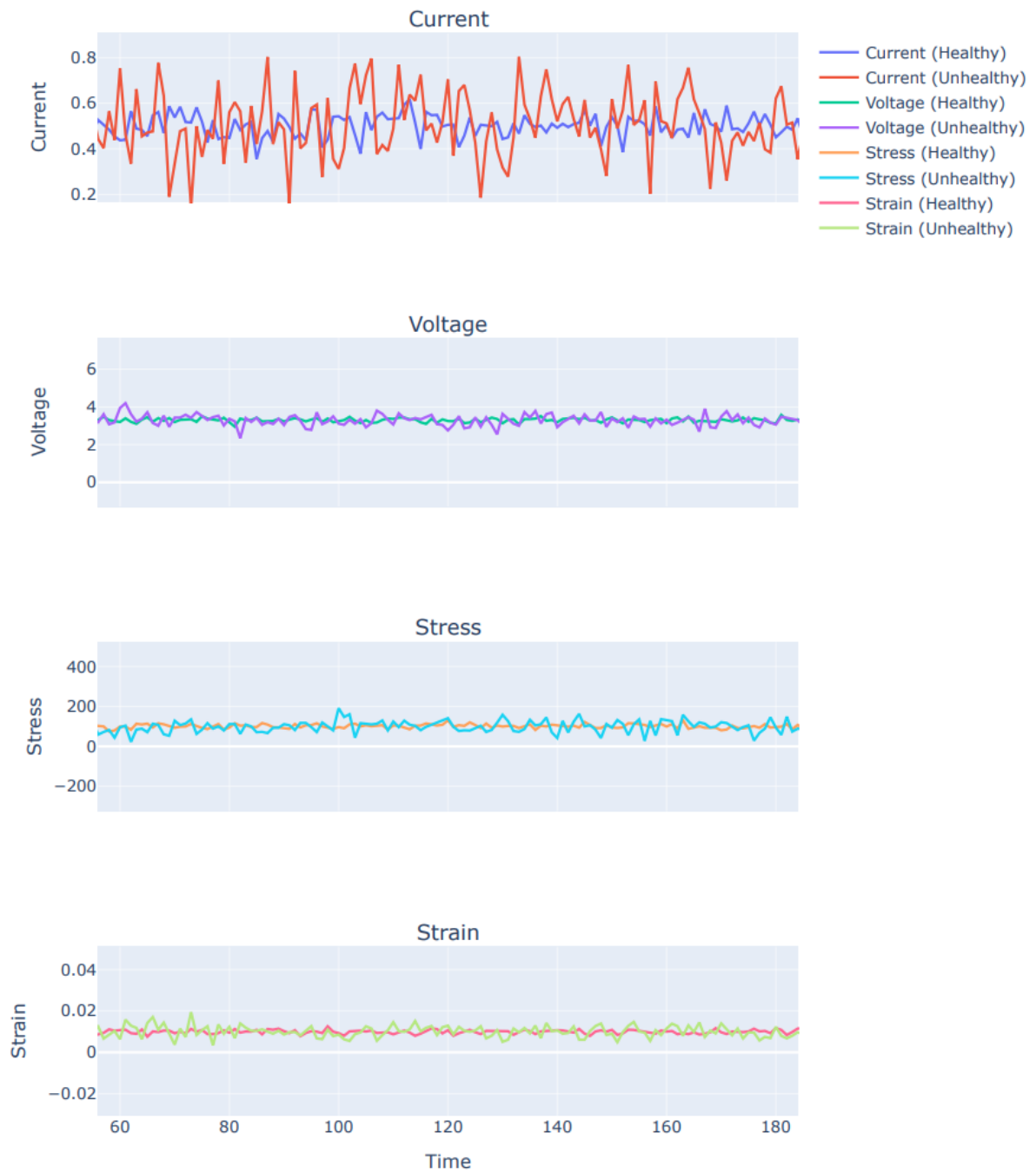
Overall Analysis:

- Comparing these histograms for healthy data from both Isolation Forest and LOF algorithms helps assess how effectively each algorithm identifies normal data.

- Differences in score distributions highlight each method's ability to recognize healthy data and can guide the adjustment of anomaly detection thresholds.
- Setting appropriate thresholds based on these distributions will aid in optimizing the models for better detection of anomalies in both healthy and unhealthy satellite structure data.

Plots of Mechanical Properties

Time Series of Features (Healthy vs Unhealthy Data)



Explantion-

The above is the time series plots for various features (Current, Voltage, Stress, and Strain) for both healthy and unhealthy structures.

- It compares the difference in the behavior of both healthy and unhealthy structures, henc making easy to find any anomaly

Current:

- Current (Healthy): Represented by the blue line.
- Current (Unhealthy): Represented by the red line.
- Observation: The current for the unhealthy structure exhibits more fluctuations and higher peaks compared to the healthy structure, indicating potential irregularities or faults in the system.

Voltage:

- Voltage (Healthy): Represented by the cyan line.
- Voltage (Unhealthy): Represented by the purple line.
- Observation: The voltage readings for both healthy and unhealthy structures are relatively stable with minor deviations. However, slight differences in the voltage levels can be observed, which may indicate issues in the unhealthy structure.

Stress:

- Stress (Healthy): Represented by the orange line.
- Stress (Unhealthy): Represented by the blue line.
- Observation: The stress levels for both healthy and unhealthy structures show some variance. The unhealthy structure has more noticeable peaks and fluctuations, which could be a sign of stress concentration and potential structural issues.

Strain:

- Strain (Healthy): Represented by the pink line.
- Strain (Unhealthy): Represented by the green line.

- Observation: The strain values for both healthy and unhealthy structures are relatively close to each other, but the unhealthy structure shows slightly more variation, suggesting the presence of strain anomalies.

General Observations:

- Healthy Data: Characterized by smoother and more stable lines, indicating consistent and expected behavior of the structure.

- Unhealthy Data: Exhibits more fluctuations, higher peaks, and greater variance, pointing towards potential faults, anomalies, or damage in the structure.